

PCDVQ report

Li Daniil

Skoltech

2025

Polar Coordinate Decoupling Vector Quantization (PCDVQ)

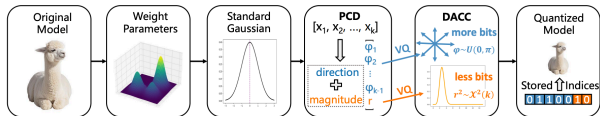


Figure 1: The pipeline of PCDVQ

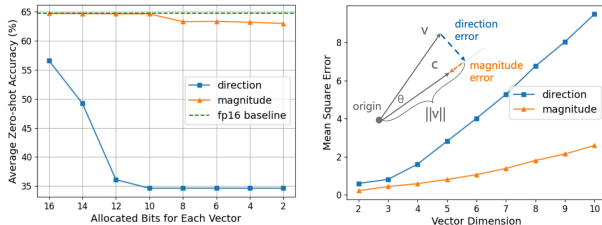


Figure 2: Compare direction and magnitude

This method consist of the next stages (Fig. 1):

- Standard Gaussian Regularization (SGR). They used Random Hadamard Transformation (RHT) for SGR
- Polar Coordinate Decoupling (PCD)
- Distribution Aligned Codebook Construction (DACC)

Motivation: direction is identified to be more essential than magnitude (Fig. 2)

Implementation

Implementation: <https://github.com/danlee65071/pcdvq/tree/main>

- ▶ Model: Qwen3-4B
- ▶ Dataset: WikiText2

The approach was tested with $a = 5$, $b = 2$ (bits for direction/magnitude). Authors used $a \in \{14, 16\}$ and $b = 2$ (Fig. 3).

$k = 64$ was used for the first reshape (original weights) and $k = 8$ for directions. In the paper k for weights was 8 (Fig. 3).

A.3 Bit Setting

The bit here denotes the Bits Per Weight (BPW). After quantization, the vector is represented by a index of direction and a index of magnitude. For PCDVQ at 2.125-bit, we set a to 16 and b to 2. As we set the vector dimension k to 8, the BPW can be calculated by $(a + b)/k = 2.125$. Notably, the bits consumption of codebooks are shared by the total model with billions of positions, which is minimal enough to be omitted.

Figure 3: From the paper: bit settings

Implementation of the fast Walsh Hadamard transform (FWHT)

Algorithm 1 Fast Walsh–Hadamard Transform

```
1: procedure fwht( $a$ )
2:    $n \leftarrow |a|$ , assert  $n$  — power of 2
3:    $h \leftarrow 1$ 
4:   while  $h < n$  do
5:     for  $i \leftarrow 0$ ;  $i < n$ ;  $i \leftarrow i + 2h$  do
6:       for  $j \leftarrow i$ ;  $j < i + h$ ;  $j \leftarrow j + 1$  do
7:          $x \leftarrow a[j]$ ;  $y \leftarrow a[j + h]$ 
8:          $a[j] \leftarrow x + y$ ;    $a[j + h] \leftarrow x - y$ 
9:       end for
10:    end for
11:    divide  $a[\cdot]$  by  $\sqrt{2}$ ;    $h \leftarrow 2h$ 
12:  end while
13: end procedure
```

A random Hadamard transformation is used to bring the weights to SGR. **Fast Walsh–Hadamard transform (FWHT)** is an efficient algorithm to compute the Walsh–Hadamard transform (WHT). This algorithm has a complexity of $O(n \log n)$ (Algorithm 1).

The **hadamard-transform** python package was used (<https://github.com/amitport/hadamard-transform>). Forward and reverse applications are identical

Implementation of the polar coordinates decoupling (PCD). To polar

```
1 def to_polar(x: torch.Tensor) -> Tuple[torch.  
    Tensor, torch.Tensor]:  
2     p, k = x.shape  
3     magnitudes = torch.linalg.vector_norm(x, dim  
        =1, keepdim=True)  
4     phis = torch.empty(p, k-1, dtype=x.dtype,  
        device=x.device)  
5     x_squares = x * x  
6     x_squares_flipped = torch.flip(x_squares,  
        dims=[1])  
7     x_squares_cumsum_flipped = torch.cumsum(  
        x_squares_flipped, dim=1)  
8     x_squares_cumsum = torch.flip(  
        x_squares_cumsum_flipped, dims=[1])  
9     y_head = x_squares_cumsum[:, 1:-1].sqrt()  
10    x_head = x[:, :-2]  
11    phis[:, :-1] = torch.atan2(y_head, x_head)  
12    phi_last = torch.atan2(x[:, -1], x[:, -2])  
13    phi_last = (phi_last + 2 * math.pi) % (2 *  
        math.pi)  
14    phis[:, -1] = phi_last  
15    return phis, magnitudes.squeeze(-1)
```

The following transformation to spherical coordinates was implemented

This function converts the vector representation x from Cartesian coordinates to polar coordinates: it calculates the norm (modulus) of each vector and the angles between the axes using arctangents. For the last two coordinates, the angle is additionally adjusted so that it always lies in the range $(0, 2\pi]$

Implementation of the polar coordinates decoupling (PCD). To cartesian

```
1 def get_unit_directions(directions: torch.Tensor
2   ) -> torch.Tensor:
3     sin_matrix = torch.sin(directions)
4     cos_matrix = torch.cos(directions)
5     cum_sin = torch.cumprod(sin_matrix, dim=-1)
6     sin_prefix = torch.cat([torch.ones_like(
7         sin_matrix[:, :1]), cum_sin[:, :-1]],
8         dim=-1)
9     x_except_last = sin_prefix * cos_matrix
10    x_last = cum_sin[:, -1:].clone()
11    unitary_directions = torch.cat([
12        x_except_last, x_last], dim=-1)
13    return unitary_directions
14
15 def to_cartesian(phis: torch.Tensor, r: torch.
16   Tensor) -> torch.Tensor:
17     if r.ndim == 1:
18         r = r.unsqueeze(-1)
19     unit_dirs = PCDVQ.get_unit_directions(phis)
20     x = r * unit_dirs
21     return x
```

get_unit_directions: constructs a unit direction vector based on sets of angles in hyperspherical coordinates. Takes cos for each axis, accumulates the products of sin from left to right, and obtains the components as $\sin_prefix \cdot \cos$, and the last one as the full product of sin

to_cartesian: onverts the radius r to a column if necessary, calculates unit directions via *get_unit_directions(phis)*, and multiplies them by r

E_8 lattice

```
1 def e8_minimal_directions(device=None, dtype=
    torch.float16, normalize=False):
2     eye_matrix = torch.eye(8, dtype=dtype,
        device=device)
3     row_idxes, col_idxes = torch.triu_indices(8,
        8, offset=1, dtype=torch.int64, device=
        device)
4     e1, e2 = eye_matrix[row_idxes], eye_matrix[
        col_idxes]
5     int_roots = torch.cat([e1 + e2, e1 - e2, -e1
        + e2, -e1 - e2], 0)
6     signs = torch.tensor([-1.0, 1.0], dtype=
        dtype, device=device)
7     halves = torch.cartesian_prod(*([signs] * 8))
        .reshape(-1, 8)
8     half_roots = 0.5 * halves[(halves.prod(dim=-1)
        > 0)]
9     e8_roots = torch.cat([int_roots, half_roots
        ], dim=0)
10    if normalize:
11        e8_roots = e8_roots / math.sqrt(2.0)
12    return e8_roots
```

The E_8 lattice can be described as all points in $\mathbb{R}^8$ such that either all coordinates are integers or all are half-integers, and the sum of coordinates is even:

$$E_8 = \left\{ (x_i) \in \mathbb{Z}^8 \cup (\mathbb{Z} + \frac{1}{2})^8 \mid \sum_i x_i \equiv 0 \pmod{2} \right\}.$$

Direction codebook

```
1 def construct_direction_codebook(a, seed=42, dtype=
    torch.float32, device=device):
2     num_centers = 2 ** a
3     directions = e8_minimal_directions(dtype, device
        )
4     g = None
5     if seed is not None:
6         g = torch.Generator(device=device)
7         g.manual_seed(seed)
8     rand_candidate_idx = torch.randint(0, directions
        .size(0), (1,), generator=g, device=device).
        item()
9     candidates_idx = [rand_candidate_idx]
10    for _ in range(1, num_centers):
11        codebook = directions[candidates_idx]
12        sims = directions @ codebook.T
13        max_sim, _ = sims.max(dim=1)
14        max_sim[candidates_idx] = torch.finfo(dtype)
            .max
15        next_candidate_idx = torch.argmin(max_sim).
            item()
16        candidates_idx.append(next_candidate_idx)
17    return directions[candidates_idx]
```

This algorithm iteratively adds to the codebook the vector that has the minimum cosine proximity to those already selected (farthest-point sampling according to the cosine metric), thereby distributing the directions as evenly as possible, and returns the selected directions.

Magnitude codebook

Algorithm 2 Construction of the magnitude codebook via Lloyd–Max iterations

```
1: procedure magnitudeCodebook( $b, k, \tau, tol, max\_iters$ )
2:    $n \leftarrow 2^b$ 
3:    $r_{\max} \leftarrow \text{FindMaxRBisection}(k, \tau)$ 
4:    $c \leftarrow \text{Midpoints}(\text{Linspace}(0, r_{\max}, n+1))$ 
5:   for  $t \leftarrow 1$  to  $max\_iters$  do
6:      $u_0 \leftarrow 0, u_n \leftarrow r_{\max}; \quad u_i \leftarrow (c_i + c_{i+1})/2$ 
7:      $N \leftarrow \text{gammainc}(\frac{k+1}{2}, \frac{u_i^2}{2}) - \text{gammainc}(\frac{k+1}{2}, \frac{u_{i-1}^2}{2})$ 
8:      $D \leftarrow \text{gammainc}(\frac{k}{2}, \frac{u_i^2}{2}) - \text{gammainc}(\frac{k}{2}, \frac{u_{i-1}^2}{2})$ 
9:      $c'_i \leftarrow \sqrt{2} \frac{\Gamma(\frac{k+1}{2})}{\Gamma(\frac{k}{2})} \cdot \frac{N}{D + \varepsilon}$ 
10:    if  $\max_i |c'_i - c_i| < tol$  then
11:      break
12:    end if
13:     $c \leftarrow c'$ 
14:  end for
15:  return  $c$ 
16: end procedure
```

Performs Lloyd–Max iterations: recompute bin borders u_i and update each center as the conditional expectation of radius in its bin (via incomplete gamma for χ_k), until $\max |c' - c| < tol$.

$$c'_i = \sqrt{2} \frac{\Gamma(\frac{k+1}{2})}{\Gamma(\frac{k}{2})} \frac{P(\frac{k+1}{2}, \frac{u_i^2}{2}) - P(\frac{k+1}{2}, \frac{u_{i-1}^2}{2})}{P(\frac{k}{2}, \frac{u_i^2}{2}) - P(\frac{k}{2}, \frac{u_{i-1}^2}{2}) + \varepsilon}.$$

Results

- ▶ Mean PPL without quantization: 425.5
- ▶ Mean PPL with quantization: 151935.5

Problems:

- ▶ After quantization, the weight matrix consists almost entirely of 0 (most likely after direction dequantization)
- ▶ After quantization, most values are replaced with one

Plans:

- ▶ Analyze E_8 lattice and magnitude codebook building
- ▶ Add more bits for directions
- ▶ Provide fine tuning (authors did it)